



Fullstack Web Development

Session 8 | Week 8/Day 1 | 120 min

Laravel - Part01 - PHP

Instructor: Ms. Liev Nova



090667393



Content

- **Introduction to PHP**
- Basic PHP

What You Should Already Know

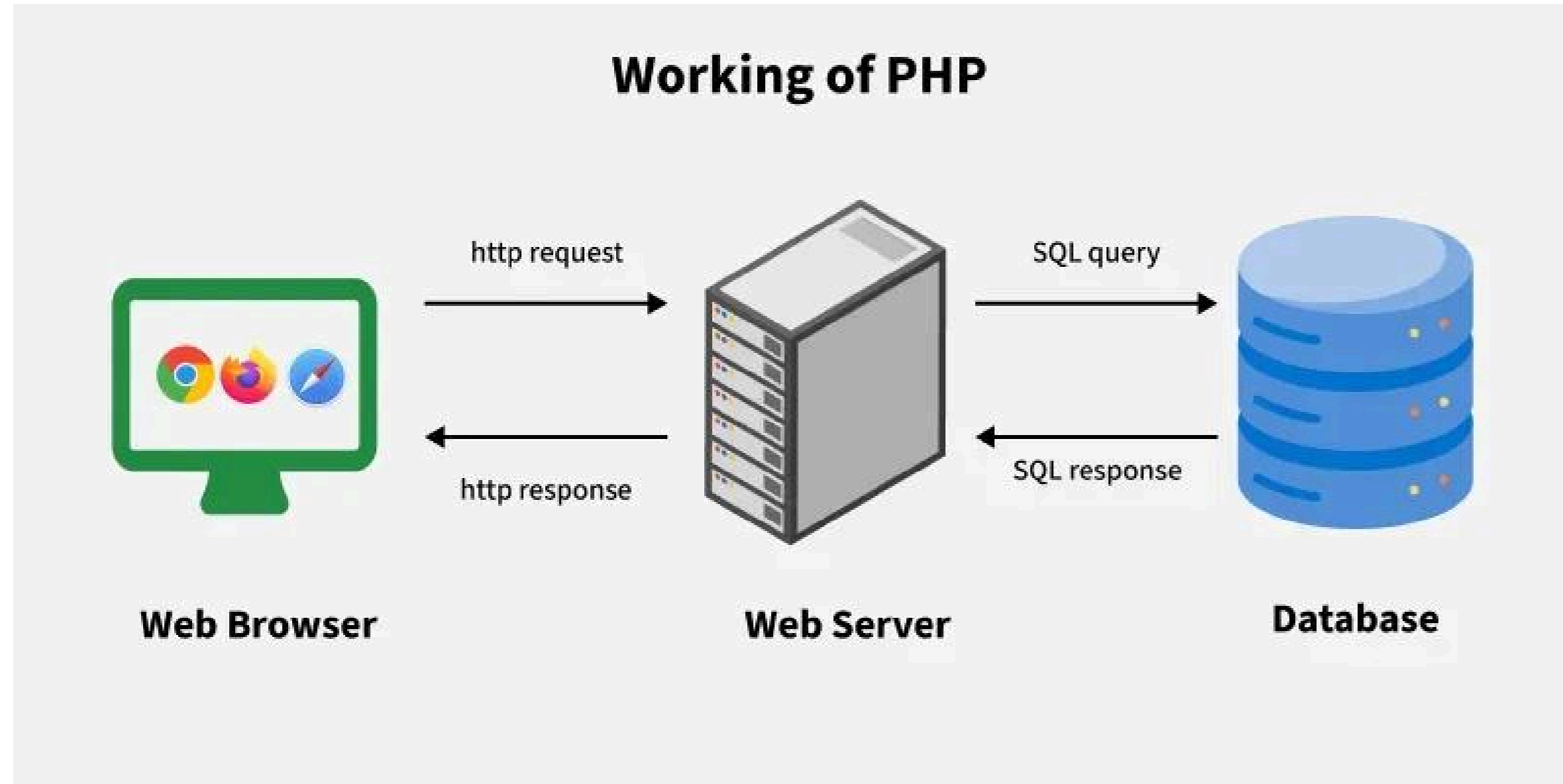


What is PHP?

- **PHP** stands for “**PHP: Hypertext Preprocessor**”
- It is a **server-side scripting language**
- PHP is **widely used** for web development
- It is **open-source** (free to use and download)
- PHP runs on a **web server** (not in the browser)

Example: When a user visits a website, PHP processes the request on the server and sends back the result.

What is PHP?



What is PHP?

Key Features of PHP:

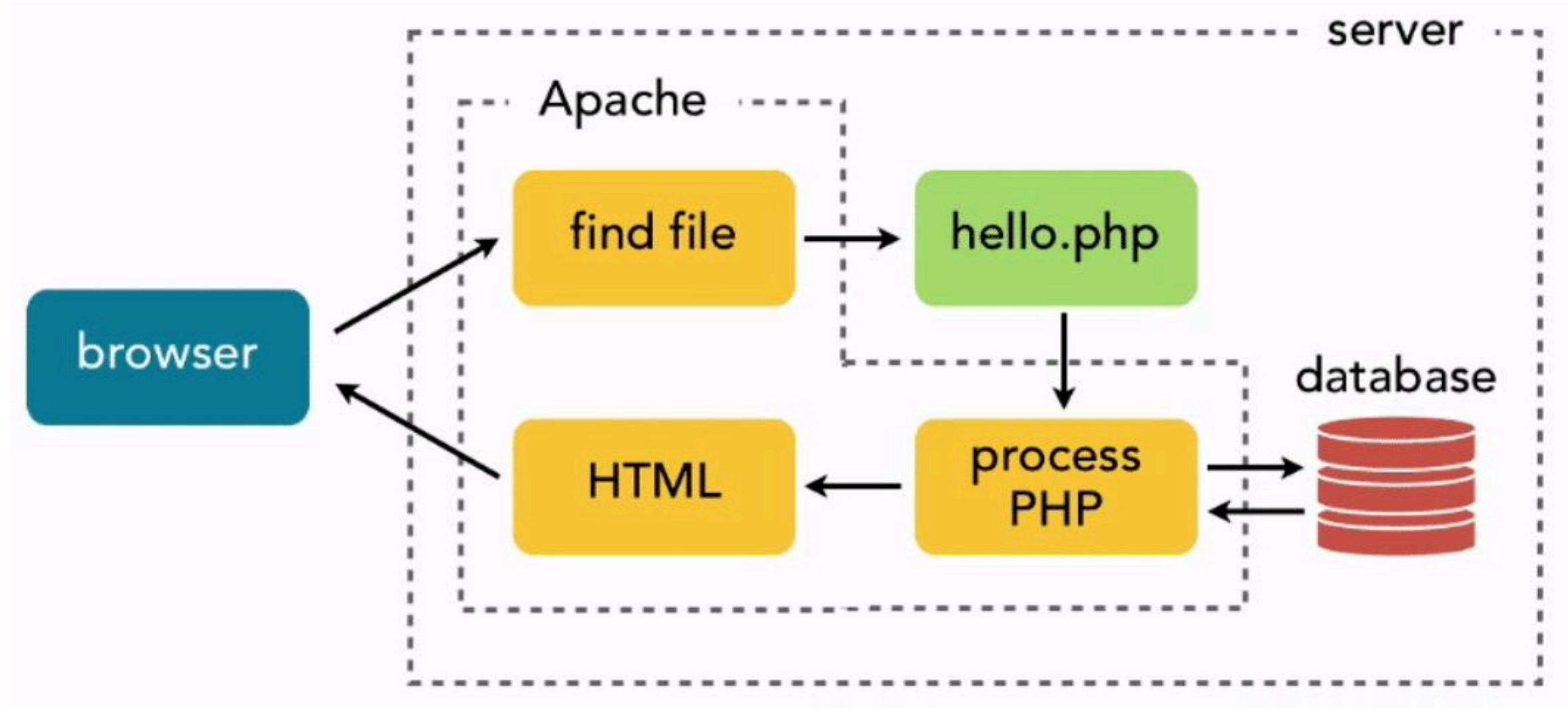
- Easy to learn and use
- Works with HTML easily
- Supports many databases (MySQL, PostgreSQL, etc.)
- Cross-platform (Windows, Linux, macOS)
- Fast and efficient for web applications

How PHP Works

- User requests a web page (index.php)
- The server executes **PHP code**
- PHP generates **HTML output**
- The browser displays the result

Users never see PHP code — only the output (HTML)

How PHP Works



What is a PHP File?

- A PHP file can contain:
 - HTML
 - CSS
 - JavaScript
 - PHP code
- PHP files:
 - Are executed on the server
 - Return plain HTML to the browser
- File extension: **.php**
- **PHP** code is written inside:

```
<?php
// PHP code here
?>
```
- Built-in PHP Server: *php -S localhost:8000*

```
<!DOCTYPE html>
<html>

<body>

    <h1>My First PHP Page</h1>

    <?php
        echo "Hello, World!";
    ?>

</body>

</html>
```

My First PHP Page

Hello, World!

What Can PHP Do?

- **Server-Side Features**

- Generate dynamic web pages
- Handle form data
- Manage sessions and cookies
- Control user authentication & access
- Work with databases (MySQL,...)
- Perform file operations:
 - Create
 - Read
 - Update
 - Delete
- Encrypt and secure data

- **Database Operations**

- Insert data
- Update records
- Delete data
- Fetch/display data
- Example:

```
$conn = mysqli_connect("localhost", "root", "", "test");
```

What Can PHP Do?

- **Cookies & Sessions**

- Store user data temporarily
- Maintain login sessions

- **PHP as Command-Line Tool**

- PHP is not only for websites!
- You can run PHP in terminal: *php script.php*

- **Summary**

- PHP is a server-side scripting language
- Used to create dynamic web applications
- Runs on the server, not the browser
- Supports database, files, sessions, security
- Can also run in command line

- **Used for:**

- Automation scripts
- System tasks (CRON jobs)
- Data processing
- Log analysis

Content

- Introduction to PHP
- **Basic PHP**

Basic PHP Syntax

- A PHP script can be placed **anywhere** in an HTML document

```
<?php
    // PHP code here
?>
```

```
<!DOCTYPE html>
<html>

<body>

    <?php
    echo "Hello World!<br>";
    echo "Hello World!<br>";
    echo "Hello World!<br>";
    ?>

</body>

</html>
```

Hello World!
Hello World!
Hello World!

Basic PHP Syntax

- **Case Sensitivity in PHP**

- Keywords are NOT case-sensitive

```
<?php
ECHO "Hello<br>";
echo "Hello<br>";
EcHo "Hello<br>";
?>
```

All work (keywords)

- But variables ARE case-sensitive

```
<?php
$age = 20;
$AGE = 30;

echo $age; // 20
echo $AGE; // 30
?>
```

\$age and \$AGE are different variables

PHP Variables

- **What is a Variable?**

- A variable stores data.

```
<?php
$x = 5;
$y = 4;

echo $x + $y;
?>
```

- **Rules for PHP Variables**

- Must start with \$
- Must start with a letter or underscore
- Cannot start with a number
- Can contain letters, numbers, underscores
- Case-sensitive
 - Example:

```
$age = 25;           // valid
$_name = "Nova";    // valid
$1value = 100;      // invalid
```

PHP echo and print Statements

- Both are used to display output
 - **echo** has no return value while **print** has a return value of 1 so it can be used in expressions.
 - **echo** can take multiple parameters (although such usage is rare) while **print** can take one argument.
 - **echo** is marginally faster than print.

```
<?php
$txt1 = "Learn PHP";
$txt2 = "with ITED";
$x = 5;
$y = 4;

echo "<h2>" . $txt1 . "</h2>";
echo "Study PHP at " . $txt2 . "<br>";
echo $x + $y;
?>
```


PHP Data Type

- **What Are Data Types?**
 - A data type defines the kind of data a variable can store.
- **PHP supports the following data types:**
 - String
 - Integer
 - Float (floating point numbers - also called double)
 - Boolean
 - Array
 - Object
 - NULL

PHP STRINGS

- **What is a String?**

- A string is a sequence of characters (text)
- Example:

```
<?php  
echo "Hello World!";  
?>
```

- **Important Notes:**

- Strings are written inside:
 - "double quotes"
 - 'single quotes'
- PHP string functions are built-in
 - No installation required

PHP STRINGS

- PHP String Functions

Function	Description	Example Code	Output
echo()	Output one or more strings	echo "Hello"; echo " World";	Hello World
print()	Output a string	print "Hello PHP";	Hello PHP
chr()	Convert ASCII to character	echo chr(65);	A
substr()	Get part of a string	echo substr("Hello World", 0, 5);	Hello
substr_compare()	Compare strings	echo substr_compare("Hello World", "World", 6);	0 (<i>match</i>)
trim()	Remove spaces	echo trim(" Hello ");	Hello

PHP NUMBERS

- **What Are PHP Numbers?**
 - PHP supports different types of numbers:
 - **Integer** → Whole numbers
 - **Float (Double)** → Decimal numbers
 - **Number Strings** → Numbers stored as text
- **Automatic Type Conversion**
 - PHP can automatically convert data types when needed.

```
<?php
    $x = "10";
    $y = 5;

    echo $x + $y; // PHP converts string "10" to number
?>
```

```
<?php
// Cast float to int
$x = 23465.768;
$int_cast = (int)$x;
echo $int_cast;

echo "<br>";

// Cast string to int
$x = "23465.768";
$int_cast = (int)$x;
echo $int_cast;
?>
```

PHP NUMBERS

- **Types of Numbers in PHP**

Type	Description	Example Code	Output
Integer	Whole numbers (no decimal)	<code><?php \$x = 100; var_dump(\$x); ?></code>	int(100)
Float (Double)	Numbers with decimals	<code><?php \$x = 10.5; var_dump(\$x); ?></code>	float(10.5)
Number String	Numbers written as text	<code><?php \$x = "12345"; var_dump(\$x); ?></code>	string(5) "12345"

- **PHP Casting Numbers**

- Casting = converting one data type to another

Conversion	Description	Example Code	Output
Float → Integer	Remove decimal part	<code><?php \$x = 23465.768; echo (int)\$x; ?></code>	23465
String → Integer	Convert string to number	<code><?php \$x = "23465.768"; echo (int)\$x; ?></code>	23465

PHP NUMBERS

- **Important Notes**

- Casting to int removes decimal part
- PHP reads numbers from left → stops at invalid character

- Example:

```
<?php
    $x = "123abc";
    echo (int)$x;
?>
```

123



PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

100



PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

100



- Integer → ?
- Float → ?
- String → ?
- Casting → ?

PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

100



- Integer → whole number
- Float → ?
- String → ?
- Casting → ?

PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

100



- Integer → whole number
- Float → decimal
- String → ?
- Casting → ?

PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

100



- Integer → whole number
- Float → decimal
- String → text number
- Casting → ?

PHP NUMBERS

- What is the output?

```
<?php
    $x = "100abc";
    echo (int)$x;
?>
```

100



- Integer → whole number
- Float → decimal
- String → text number
- Casting → change type

PHP MATH FUNCTIONS

- PHP provides built-in math functions to perform calculations easily.
- **Common PHP Math Functions**

Function	Description	Example Code	Output
pi()	Returns value of π (pi)	<code><?php echo pi(); ?></code>	3.1415926535898
min()	Returns smallest value	<code><?php echo min(0, 150, 30); ?></code>	0
max()	Returns largest value	<code><?php echo max(0, 150, 30); ?></code>	150
abs()	Returns absolute value	<code><?php echo abs(-6.7); ?></code>	6.7
sqrt()	Returns square root	<code><?php echo sqrt(64); ?></code>	8
round()	Rounds a number	<code><?php echo round(0.60); ?></code>	1
rand()	Generates random number	<code><?php echo rand(); ?></code>	Random number

PHP MATH FUNCTIONS

- PHP has built-in math functions
- No need to install anything
- Useful for calculations, random values, and data processing
- Random Number with Range
 - Example:

```
<?php  
echo rand(1, 10);  
?>
```

```
<?php  
echo "PI: " . pi() . "<br>";  
echo "Min: " . min(1, 2, 3) . "<br>";  
echo "Max: " . max(1, 2, 3) . "<br>";  
echo "Absolute: " . abs(-10) . "<br>";  
echo "Square Root: " . sqrt(25) . "<br>";  
echo "Round: " . round(4.6) . "<br>";  
echo "Random: " . rand(1, 100);  
?>
```

PHP CONSTANTS

- **What is a Constant?**

- A constant is like a variable, but:
 - Its value cannot be changed
 - It cannot be undefined
 - It does not use \$ sign

- **Creating a Constant**

- Use the `define()` function

```
<?php
    define("GREETING", "Welcome to W3Schools.com!");
    echo GREETING;
?>
```

Welcome to W3Schools.com!

PHP CONSTANTS

- **Syntax Explanation**

- `define(name, value, case_insensitive);`

Parameter	Description
name	Name of constant
value	Value of constant
case_insensitive	Optional (default: false)

- **Note:** Case-insensitive constants are deprecated (do not use).

PHP OPERATORS – Arithmetic Operators

- **What Are Arithmetic Operators?**
 - Arithmetic operators are used to perform mathematical calculations in PHP.

Operator	Name	Example	Result
+	Addition	$\$x + \y	Sum of $\$x$ and $\$y$
-	Subtraction	$\$x - \y	Difference of $\$x$ and $\$y$
*	Multiplication	$\$x * \y	Product of $\$x$ and $\$y$
/	Division	$\$x / \y	Quotient of $\$x$ and $\$y$
%	Modulus	$\$x \% \y	Remainder of $\$x \div \y
**	Exponentiation	$\$x ** \y	$\$x$ raised to power $\$y$

PHP OPERATORS – Arithmetic Operators

- **What Are Arithmetic Operators?**
 - Arithmetic operators are used to perform mathematical calculations in PHP.
 - Example:

```
<?php
    $x = 10;
    $y = 3;

    echo "Addition: " . ($x + $y) . "<br>";
    echo "Subtraction: " . ($x - $y) . "<br>";
    echo "Multiplication: " . ($x * $y) . "<br>";
    echo "Division: " . ($x / $y) . "<br>";
    echo "Modulus: " . ($x % $y) . "<br>";
    echo "Exponentiation: " . ($x ** $y);

?>
```

Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333
Modulus: 1
Exponentiation: 1000



PHP OPERATORS – Assignment Operators

- **What Are Assignment Operators?**
 - Assignment operators are used to assign values to variables

Operator	Example	Same As	Description
=	<code>\$x = \$y</code>	<code>\$x = \$y</code>	Assign value of \$y to \$x
+=	<code>\$x += \$y</code>	<code>\$x = \$x + \$y</code>	Add and assign
-=	<code>\$x -= \$y</code>	<code>\$x = \$x - \$y</code>	Subtract and assign
*=	<code>\$x *= \$y</code>	<code>\$x = \$x * \$y</code>	Multiply and assign
/=	<code>\$x /= \$y</code>	<code>\$x = \$x / \$y</code>	Divide and assign
%=	<code>\$x %= \$y</code>	<code>\$x = \$x % \$y</code>	Modulus and assign

PHP OPERATORS – Assignment Operators

- **What Are Assignment Operators?**

- Assignment operators are used to assign values to variables

- Example:

```
<?php
    $x = 10;
    $y = 5;

    $x += $y; // 10 + 5 = 15
    echo "After += : $x <br>";

    $x -= $y; // 15 - 5 = 10
    echo "After -= : $x <br>";

    $x *= $y; // 10 * 5 = 50
    echo "After *= : $x <br>";

    $x /= $y; // 50 / 5 = 10
    echo "After /= : $x <br>";

    $x %= $y; // 10 % 5 = 0
    echo "After %= : $x";

?>
```

```
After += : 15
After -= : 10
After *= : 50
After /= : 10
After %= : 0
```



PHP OPERATORS – Comparison Operators

- **What Are Comparison Operators?**
 - Comparison operators are used to compare two values
 - They always return true or false

Operator	Name	Example	Result
==	Equal	<code>\$x == \$y</code>	True if values are equal
===	Identical	<code>\$x === \$y</code>	True if value and type are equal
!=	Not equal	<code>\$x != \$y</code>	True if values are not equal
<>	Not equal	<code>\$x <> \$y</code>	Same as !=
!==	Not identical	<code>\$x !== \$y</code>	True if value or type is different
>	Greater than	<code>\$x > \$y</code>	True if \$x is greater

PHP OPERATORS – Comparison Operators

- **What Are Comparison Operators?**
 - Comparison operators are used to compare two values
 - They always return true or false

Operator	Name	Example	Result
>	Greater than	$\$x > \y	True if $\$x$ is greater
<	Less than	$\$x < \y	True if $\$x$ is smaller
>=	Greater or equal	$\$x \geq \y	True if $\$x \geq \y
<=	Less or equal	$\$x \leq \y	True if $\$x \leq \y
<=>	Spaceship	$\$x \lt=> \y	Returns -1, 0, or 1

PHP OPERATORS – Comparison Operators

- **What Are Comparison Operators?**
 - Comparison operators are used to compare two values
 - They always return true or false
 - Example:

```
<?php
    $x = 10;
    $y = 5;

    var_dump($x == $y);    // false
    var_dump($x > $y);     // true
    var_dump($x < $y);     // false
    var_dump($x >= $y);    // true
    var_dump($x <= $y);    // false
?>
```


PHP FUNCTIONS

- **What is a Function?**
 - A function is a block of code that performs a specific task
 - It can be reused multiple times
- **Syntax of a Function**

```
function functionName(){  
    // code to be executed  
}
```

- Example: Simple Function

```
<?php  
    function writeMsg(){  
        echo "Hello World!";  
    }  
    // Call the function  
    writeMsg();  
?>
```

Hello World!



PHP FUNCTIONS

- **What is a Function?**

- A function is a block of code that performs a specific task
- It can be reused multiple times

- **Syntax of a Function**

```
function functionName(){  
    // code to be executed  
}
```

```
<?php  
    function familyName($fname, $year) {  
        echo "$fname Refsnes. Born in $year <br>";  
    }  
  
    familyName("Hege", 1975);  
    familyName("Stale", 1978);  
    familyName("Kai Jim", 1983);  
?>
```

- Example: Function with Arguments

- Arguments allow passing data into a function



```
Hege Refsnes. Born in 1975  
Stale Refsnes. Born in 1978  
Kai Jim Refsnes. Born in 1983
```

PHP FUNCTIONS

- **What is a Function?**
 - A function is a block of code that performs a specific task
 - It can be reused multiple times
- **Syntax of a Function**

```
function functionName(){  
    // code to be executed  
}
```

- **Function with Return Value**

```
<?php  
    function add($x, $y){  
        return $x + $y;  
    }  
  
    echo add(5, 3);  
?>
```

8



PHP FUNCTIONS

- **What is a Function?**
 - A function is a block of code that performs a specific task
 - It can be reused multiple times
- **Summary**

Feature	Description
Function	Reusable block of code
Arguments	Input values
Return	Output value
Call	Run the function

PHP ARRAYS

- **What is an Array?**

- An array is a variable that can store multiple values in a single variable

- Example of Array.

```
<?php
    $cars = array("Volvo", "BMW", "Toyota");

    echo "I like " . $cars[0] . ", " . $cars[1] . " and " . $cars[2] . ".";
?>
```



I like Volvo, BMW and Toyota.

PHP ARRAYS

- **Types of Arrays in PHP**

Type	Description
Indexed Array	Uses numeric index (0,1,2...)
Associative Array	Uses named keys
Multidimensional Array	Array inside another array

PHP ARRAYS

- Associative Array Example

```
<?php
    $age = array("Peter" => 35, "Ben" => 37);

    echo "Peter is " . $age["Peter"] . " years old.";
?>
```

Peter is 35 years old.



- Multidimensional Array Example

Volvo



```
<?php
    $cars = array(
        array("Volvo", 22),
        array("BMW", 15)
    );

    echo $cars[0][0]; // Volvo
?>
```

PHP GLOBAL VARIABLES – SUPERGLOBALS

- **What Are Superglobals?**
 - **Superglobals** are built-in variables in PHP
 - They are **accessible from anywhere** (global scope)
 - can use them in:
 - Functions
 - Classes
 - Files
 - No need to declare **global**

PHP GLOBAL VARIABLES – SUPERGLOBALS

Superglobal	Description
<code>\$GLOBALS</code>	Access global variables
<code>\$_SERVER</code>	Server and execution info
<code>\$_REQUEST</code>	Collect form data (GET, POST, COOKIE)
<code>\$_POST</code>	Collect data from POST method
<code>\$_GET</code>	Collect data from URL (GET method)
<code>\$_FILES</code>	Handle file uploads
<code>\$_ENV</code>	Environment variables
<code>\$_COOKIE</code>	Store user data in browser
<code>\$_SESSION</code>	Store user data across pages



THANK YOU

For your attention !

